

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа

Выполнила:

Машковцева Марина

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Опишите все функциональные и нефункциональные требования к вашему бэкенд-приложению.
2. Выберите формат реализации API, которого вы будете придерживаться: REST API или Backend For Frontend.
3. Спроектируйте все эндпоинты вашего API в формате OpenAPI-схемы с учётом реализованной диаграммы БД.
4. Опишите тело каждого запроса и тело каждого ответа, продумайте возможные ошибки, возвращаемые из API.

Вариант 5. Сервис для обмена рецептами и кулинарных блогов

- Вход
- Регистрация
- Личный кабинет пользователя (сохраненные рецепты, публикации)
- Поиск рецептов с фильтрацией по типу блюда, сложности, ингредиентам
- Страница рецепта с фото, пошаговыми инструкциями и видео
- Социальные функции (комментарии, лайки, подписки на кулинаров)

Ход работы

Функциональные требования

Разрабатываемое бэкенд-приложение должно обеспечивать следующие функции:

- Регистрация и аутентификация пользователей
- Просмотр и редактирование профиля пользователя
- Создание, просмотр и удаление рецептов
- Поиск рецептов с фильтрацией по категории, сложности и ингредиентам
- Просмотр подробной информации о рецепте (описание, шаги, медиа)
- Добавление комментариев к рецептам
- Возможность ставить лайки рецептам
- Возможность сохранять рецепты в личный кабинет
- Возможность подписываться на других пользователей

Нефункциональные требования

- Приложение реализовано в виде REST API
- Обмен данными осуществляется в формате JSON
- Используются стандартные HTTP-методы (GET, POST, DELETE)
- Обработка ошибок с использованием HTTP-кодов (400, 401, 403, 404, 500)
- Обеспечение масштабируемости и расширяемости системы
- Поддержка модульной структуры приложения
- Обеспечение безопасности (аутентификация пользователей)

Выбор архитектуры API

В рамках задания был выбран подход REST API.

REST API использует стандартные HTTP-методы и хорошо подходит для веб- и мобильных приложений.

Проектирование API

Для описания API был выбран язык TypeSpec. Он позволяет описывать API в декларативном стиле с использованием TypeScript-подобного синтаксиса, после чего автоматически генерировать спецификацию OpenAPI 3.0.

Описание запросов и ответов

Для каждого эндпоинта были определены:

- Тело запроса (request body) в формате JSON
- Параметры пути (path parameters) и query-параметры
- Тело ответа (response body)
- Возможные коды ошибок

Основные коды ответов:

- 200 – успешное выполнение запроса
- 201 – ресурс успешно создан
- 400 – ошибка в данных запроса
- 401 – пользователь не авторизован
- 403 – недостаточно прав
- 404 – ресурс не найден

- 500 – внутренняя ошибка сервера

Генерация документации

Для автоматической генерации документации API была настроена компиляция TypeSpec в спецификацию OpenAPI 3.0. Команда компиляции:

```
npx tsp compile main.tsp --config tspconfig.yaml
```

Сгенерированная спецификация в формате YAML располагается по пути:

```
tsp-output/@typespec/openapi3/openapi.yaml
```

Для визуализации документации был развернут Swagger UI на локальном сервере. Просмотр документации осуществляется по адресу:

```
http://localhost:3000/api-docs
```

Swagger UI позволяет не только просматривать структуру API, но и тестировать эндпоинты в интерактивном режиме, отправляя реальные запросы к API.

Вывод

В результате был разработан и задокументирован REST API для сервиса обмена рецептами. API полностью соответствует разработанной ранее структуре базы данных и покрывает все основные функции системы.

Для визуализации и тестирования API используется Swagger, что позволяет удобно взаимодействовать с эндпоинтами и демонстрировать работу приложения.